Backend & Distributed Systems from First Principles

Interview study guide for **MeshPay** (upi-offline-mesh), a Spring Boot backend for offline UPI payments routed through a Bluetooth-style mesh. Source: github.com/shaswatnaman/upi-offline-mesh

How this was written. The code was read from the GitHub copy of `main` (same repo you cloned): the README, `HybridCryptoService`, `IdempotencyService`, `BridgeIngestionService`, `SettlementService`, `MeshSimulatorService` and `IdempotencyConcurrencyTest`. Statements about `Account`, `Transaction` and `ApiController` come from the README, not the source.

1. The problem from first principles

Alice is offline and wants to pay Bob. Her phone hands the payment to strangers, and eventually one stranger with internet uploads it. That setup gives four facts to design around:

1. **The carriers are untrusted.** They can read, modify, copy or replay the packet.
2. **Delivery is unreliable and duplicated.** Many carriers hold the same packet, so the server may receive it many times, possibly at the same instant.
3. **Time has passed.** The packet may arrive hours or days after it was created.
4. **Money must move exactly once.**

Each fact maps to a mechanism in the code:

Problem	Mechanism	Class
Untrusted carriers	Hybrid encryption (RSA-OAEP + AES-GCM)	HybridCryptoService
Duplicates and races	Atomic idempotency claim	IdempotencyService
Old or replayed packets	Timestamp inside the encrypted payload	BridgeIngestionService
Exactly-once money movement	DB transaction + optimistic locking	SettlementService
Packets spreading offline	Gossip flooding with TTL	MeshSimulatorService

> **The sentence to memorize:** the network gives you *at-least-once* delivery, and an idempotent consumer turns that into *effectively-once* processing. Exactly-once delivery is impossible over an unreliable network. What you build instead is "duplicates are harmless."

2. Gossip and flooding (the mesh)

In an ad-hoc network no node knows the topology, so there is no routing table. The simplest strategy is to tell everyone you meet. That is an **epidemic or gossip protocol**, and it has three properties:

- **Robust.** Any path that exists will probably be found.
- **Wasteful.** One packet can generate on the order of n^2 transfers.
- **Unbounded without a limit.** Packets would circulate forever, so each copy carries a **TTL (time-to-live)**. It drops by one per hop, and at 0 the copy stops forwarding.

In `gossipOnce()` three details are worth knowing:

- It **snapshots** what each device holds before the round starts. Without that, a packet would cascade through every device in one step and the meaning of "per round" would disappear.
- It deduplicates **per device by packetId** (`dst.holds(...)`). That is the mesh's own, weaker form of duplicate suppression.
- The shared device map is a `ConcurrentHashMap`, since HTTP threads and the demo flush run concurrently.

This design is **store-and-forward**, also called delay-tolerant networking: a node holds data until it meets a link. The bridge node is the link to the internet.

> **Why this matters:** flooding guarantees duplicates. Idempotency is therefore required, not optional.

3. Cryptography: confidentiality and integrity

Symmetric vs asymmetric

- **Symmetric (AES)** uses one shared key. It is fast and handles any size, but you must share the key somehow.
- **Asymmetric (RSA)** encrypts with a public key and decrypts with a private key. It needs no shared secret, but it is slow and can only encrypt a small block.

Why hybrid

Use asymmetric crypto to move a small secret (the AES key), then symmetric crypto for the bulk data. TLS, PGP and Signal all do this. The wire format in this project is:

```
[256 B RSA-encrypted AES key][12 B IV][AES ciphertext + 16 B GCM tag]
```

A 2048-bit RSA output is always 256 bytes, which is where the 256 in `decrypt()` comes from.

> **An accuracy catch for interviews:** the code comment says RSA can encrypt about 245 bytes. That is the limit for the older PKCS#1 v1.5 padding. With OAEP and SHA-256 the limit is $256 - 2 \times 32 - 2 =$ **190 bytes**. The conclusion (JSON may not fit, so go hybrid) is unchanged, but the number is wrong, and being right about it signals depth.

RSA-OAEP

Textbook RSA is deterministic and malleable. **OAEP** adds random padding, so encrypting the same message twice gives different ciphertexts, and it makes chosen-ciphertext attacks impractical.

AES-GCM (authenticated encryption, "AEAD")

GCM does two jobs at once:

- **Counter mode** produces a keystream that is XORed with the plaintext. This gives confidentiality.
- **GHASH** computes a 128-bit authentication tag over the ciphertext. This gives integrity.

On decryption the tag is verified first. Flip any bit and `doFinal` throws, so you never get corrupted plaintext. That is why `tamperedCiphertextIsRejected` passes: the result is `INVALID / decryption_failed`.

The rule that makes GCM safe: never reuse an (key, IV) pair. Here a fresh random AES key is generated per packet, so IV reuse is not a risk. That is a deliberate design strength.

SHA-256 as the idempotency key

`hashCiphertext` produces a fixed-size, collision-resistant fingerprint, which makes a good map key. **Why hash the ciphertext and not packetId?** Carriers can rewrite packetId, but they cannot produce a valid ciphertext for a different payload. Copies of the same packet are byte-identical, so their hashes match.

A README claim to be careful with: it says identical ciphertexts come from "same key + IV + plaintext." In the code the key and IV are random on every `encrypt()`. The copies match because they are *copies of the same bytes*, not re-encryptions.

The gap interviewers will probe

Confidentiality and integrity are not **sender authentication**. The server's public key is public (`GET /api/server-key`), so anyone can build a valid packet claiming to be Alice. `PaymentInstruction` has a `pinHash` but no digital signature. The fix is for the sender's device to **sign** the payload (ECDSA or Ed25519) and for the server to verify it against a registered device key. The README lists device certificates as production work.

4. Idempotency and concurrency (the core)

Idempotent means doing it twice has the same effect as doing it once. The **idempotency key** identifies "the same request"; here it is the ciphertext hash.

The race you must be able to draw

A naive check-then-act:

```
if (!seen.containsKey(h)) {   // thread A and B both see "absent"
    seen.put(h, now);         // both proceed -> double debit
    settle();
}
```

Between the check and the write, another thread can slip in. This is a **TOCTOU** race (time-of-check to time-of-use). The fix is to make check and write **one atomic operation**:

```
return seen.putIfAbsent(packetHash, Instant.now()) == null;
```

`putIfAbsent` returns the previous value, so exactly one caller gets `null`. Under the hood, `ConcurrentHashMap` uses compare-and-set on an empty bucket and otherwise locks only that bucket, never the whole map. In Redis the same idea is `SET key value NX EX ttl`.

Why JVM-local is not enough

The map is correct within a single process only. With two server instances behind a load balancer, each has its own map and the race returns. Production needs a shared atomic store (Redis) or the database as arbiter. The README names Redis and says the unique index on `transactions.packet_hash` is the backstop; I am relying on the README for that index.

TTL and eviction

Without eviction the map grows forever, so a `@Scheduled` task sweeps every 60 seconds. A related subtle point: an entry is created when the packet is first *seen*, which is at or after `signedAt`. So the cache entry always outlives the packet's 24-hour freshness window, and an old replay hits either the cache or the staleness check.

The test

`CountDownLatch start` holds three threads at the gate, then `countDown()` releases them together. This is the standard way to maximize contention. It asserts 1 `SETTLED` and 2 `DUPLICATE_DROPPED`, then checks the balances moved exactly once.

A real flaw to know about

The claim happens **before** decrypt and settle, and nothing releases it on failure. Three consequences:

- If settlement throws (DB blip, `OptimisticLockException`), the hash stays claimed and the retry is dropped as a duplicate. **The payment is silently lost.**
- Garbage packets fill the cache, and each unique junk packet costs an expensive RSA decrypt. That is a denial-of-service surface that needs rate limiting.
- Duplicates return `DUPLICATE_DROPPED`, not the original result. Stripe-style idempotency stores the first response and replays it.

The fix is a state machine (`IN_PROGRESS → DONE / FAILED`), or inserting the ledger row with the unique key **inside** the settlement transaction so the database is the single source of truth.

5. Transactions and locking (settlement)

ACID in one paragraph

- **Atomicity:** debit, credit and ledger insert all happen or none do. `@Transactional` makes Spring wrap the method in a transaction and roll back on a runtime exception.
- **Consistency:** invariants hold (the amount is positive, the balance is never negative).
- **Isolation:** concurrent transactions do not see half-finished work.
- **Durability:** committed data survives a crash. H2 in-memory does not provide this, which is why it is a demo.

@Transactional mechanics

Spring implements it with a **proxy** (AOP). It works here because `BridgeIngestionService` calls `settlement.settle()` on an injected bean. The classic pitfall is **self-invocation**: calling a transactional method from another method in the *same* class bypasses the proxy and the transaction never starts.

The lost-update problem

Two transactions both read Alice's balance as 500 and each write back 400. One debit vanishes. Two ways to prevent it:

	Pessimistic	Optimistic (used here)
How	SELECT … FOR UPDATE locks the row	@Version column, checked at write time
Cost	Blocks other writers, can deadlock	No locks, but conflicts need a retry
Best when	High contention	Low contention

Optimistic locking in SQL terms:

```
UPDATE account SET balance = ?, version = version + 1
WHERE vpa = ? AND version = ?;   -- 0 rows updated -> OptimisticLockException
```

Pessimistic locking has a deadlock risk when A→B and B→A transfers lock in opposite orders. The fix is to always lock accounts in a consistent order, such as sorted by VPA.

Other details

- Money is `BigDecimal`, never `double`, because binary floating point cannot represent 0.1 exactly. The test compares with `compareTo`, not `equals`, because scale matters (100.0 ≠ 100.00 under equals).
- The ledger (`Transaction`) is an append-only record with a unique `packetHash`. That is the idea behind audit trails and double-entry bookkeeping.
- An insufficient balance writes a `REJECTED` row and returns normally, so it is recorded rather than thrown.

6. Replay protection and time

A replay means resending a valid old message. The defenses stack in layers:

- **signedAt inside the AEAD payload.** An attacker cannot change it without breaking the tag. The server rejects anything older than 24 hours (`stale_packet`) or more than 5 minutes in the future (`future_dated`). The 5 minutes is **clock-skew tolerance**: distributed machines never agree exactly on time, and phone clocks drift more than most.
- **Nonce (UUID)** makes two identical-looking payments produce different ciphertexts, so legitimate repeats do not dedupe against each other.
- **The idempotency cache** catches exact replays inside the window.

7. The distributed-systems theory behind it

- **CAP / partition tolerance.** The offline period *is* a network partition. The system chooses **availability**: it accepts the payment instantly on the sender's phone and reconciles later, giving up strict consistency. This is **eventual consistency** with **deferred settlement**.
- **The double-spend consequence.** Nothing checks the balance while offline, so a "sent" payment is an IOU. A malicious sender can sign two payments against the same ₹500, and whichever reaches the backend first wins. The industry fix is UPI Lite's pre-funded, hardware-attested wallet.
- **Delivery semantics.** At-most-once loses messages, at-least-once duplicates them, and "exactly-once" is really at-least-once plus idempotent processing.
- **Defense in depth.** Idempotency cache → unique DB index → optimistic lock. Each layer catches what the one above missed.

8. General backend concepts in the project

- **Layering:** controller → service → repository, with `BridgeIngestionService` as the orchestrator (a pipeline of hash → claim → decrypt → freshness → settle).
- **Dependency injection** (constructor injection) and Spring Data JPA repositories.
- **Config via @Value** with defaults (`${upi.mesh.packet-max-age-seconds:86400}`).
- **Structured outcomes:** a Java `record` with `SETTLED / DUPLICATE_DROPPED / INVALID` instead of exceptions for expected cases.

- **GlobalExceptionHandler** gives a uniform JSON error shape.
- **Tests:** `@SpringBootTest` integration tests, concurrency tests, and CI on Linux and macOS via GitHub Actions.

9. How it would scale to production

1. Redis `SET NX EX` for the idempotency claim, plus a Postgres unique index as the final arbiter.
2. Make ingest return `202 Accepted` and push packets onto a queue (Kafka or SQS). Settle asynchronously with retries and a dead-letter queue.
3. Add a transactional **outbox** so a DB commit and an event publish cannot diverge.
4. Keep the private key in an HSM or KMS, and put a **key ID** in each packet so keys can rotate.
5. Add mutual TLS or signed bridge certificates, per-bridge rate limits, and sender velocity checks.
6. Add metrics: settle, duplicate and invalid rates.

10. Interview Q&A

Q: Why hybrid encryption instead of just RSA?
RSA is slow and can only encrypt a small block (190 bytes with OAEP-SHA256 on a 2048-bit key). AES is fast and handles any size. Hybrid uses RSA only to wrap a per-packet AES key.

Q: Why GCM over CBC?
GCM is authenticated, so tampering is detected. CBC alone gives no integrity and is vulnerable to padding-oracle attacks.

Q: Why is the idempotency key the ciphertext hash?
Carriers cannot forge a ciphertext for another payload, and copies of one packet are byte-identical. packetId is rewritable, so it is untrustworthy.

Q: What happens if 100 threads claim the same hash at once?
putIfAbsent is atomic, so exactly one gets null and proceeds. The other 99 get DUPLICATE_DROPPED.

Q: Why does the idempotency check come before decrypt?
RSA decrypt is the most expensive step. A cheap hash lookup avoids repeating it for duplicates.

Q: Optimistic or pessimistic locking, and why?
Optimistic suits low contention per account. Pessimistic suits hot rows. Here optimistic is the backstop behind idempotency.

Q: What breaks with multiple server instances?
The in-memory map is per-JVM. Move the claim to Redis or the DB.

Q: What if settlement fails after the claim succeeds?
The packet is lost as a "duplicate." Fix it with an IN_PROGRESS/DONE state, or release the claim on failure.

Q: Can this prevent double-spending?
No. Offline balance cannot be verified, so a payment is an IOU until settlement. A pre-funded hardware-backed wallet is the fix.

Q: Why a 5-minute negative age tolerance?
Clock skew between the sender's phone and the server.

11. "What would you improve?" (answer these before they ask)

1. Sender signatures, so packets are authenticated and not just encrypted.
2. A claim lifecycle for the idempotency record, so failures do not lose payments.
3. A distributed idempotency store, a persistent DB, and a real KMS.
4. Auth, rate limiting and mTLS on `/api/bridge/ingest`, and the H2 console disabled.
5. Replay of the original result for duplicates, not just a drop.
6. Tests for the optimistic-lock path, which IdempotencyConcurrencyTest does not directly exercise because idempotency catches duplicates first.

Sources

github.com/shaswatnaman/upi-offline-mesh (README)

.../src/main/java/com/shaswatnaman/upimesh/crypto/HybridCryptoService.java

.../service/IdempotencyService.java, BridgeIngestionService.java, SettlementService.java, MeshSimulatorService.java

.../src/test/java/com/shaswatnaman/upimesh/IdempotencyConcurrencyTest.java